Department of Computer Science & Engineering
Indian Institute of Technology Kharagpur

Term Project Report

Course: Database Management Systems (CS30202)
Spring Semester 2025–2026

Simulation and Analysis of Buffer Manager Strategies with a Natural Language Query Interface

Authors:

Kaustav Mishra
Tanmay Amritkar
Rohit Ranjeet Satpute
Tanishq Sura
Aditya Prakash

April 14, 2026

Abstract

As the scale of data in modern applications continues to grow, disk Input/Output (I/O) remains the most significant performance bottleneck in traditional disk-resident Database Management Systems (DBMS). To mitigate latency, relational databases employ buffer pools in main memory to cache frequently accessed data pages. This project presents the design, implementation, and rigorous empirical evaluation of a complete end-to-end DBMS simulation system engineered to analyze buffer replacement algorithms. The system architecture features a decoupled, two-tier design: a C++17 backend integrating the SQLite amalgamation for high-fidelity query execution and a Python-based Streamlit frontend offering a conversational Natural Language Query interface. Inter-process communication is facilitated via shared flat files, establishing a pipeline for a locally running Llama model to perform text-to-SQL translation.

The core contribution of this research is the parallel simulation of four distinct buffer replacement strategies: Least Recently Used (LRU), Most Recently Used (MRU), CLOCK (Second-Chance), and a heuristic-based PINNED strategy. By executing a comprehensive test suite of varied relational queries against a custom educational database schema, we provide a comparative analysis of caching performance. The system demonstrates practical applications of classical database optimization techniques, highlighting the vulnerabilities of LRU to sequential flooding, the specific use-cases for MRU, the low-overhead efficiency of CLOCK, and the performance gains achievable through pinning structural data pages.

Contents

1 Introduction

1.1 Motivation

In a conventional Database Management System (DBMS), persistent data resides on secondary storage devices, such as Hard Disk Drives (HDDs) or Solid-State Drives (SSDs). Accessing these secondary storage mediums is orders of magnitude slower than accessing volatile main memory (RAM). To bridge this critical latency gap, a DBMS allocates a contiguous block of main memory known as the buffer pool. The buffer manager is the internal subsystem responsible for staging pages (fixed-size blocks of data) from disk to RAM.

When a query executes, it requests data pages. If a page is already in the buffer pool (a cache hit), it is read instantly. If it is not (a cache miss), the DBMS must execute an expensive physical disk read. When the buffer pool reaches its maximum capacity, the manager must invoke a replacement policy to evict an existing page to accommodate incoming data. The efficacy of this eviction policy directly dictates the system's disk I/O overhead and, consequently, overall query execution speed.

1.2 Problem Statement

While classical algorithms like Least Recently Used (LRU) offer strong general-case performance by exploiting temporal locality, they are highly susceptible to pathological degradation under specific database workloads. For instance, a full-table sequential scan can cause "cache thrashing" or "sequential flooding," where useful cached pages are entirely flushed by data that is only read once. Because there is no universally optimal replacement strategy, it is imperative to evaluate varying algorithms under simulated, highly-controlled query workloads to understand memory dynamics in modern database architectures.

1.3 Objectives

The primary objectives of this term project are:

1. **System Engineering:** To architect a robust, full-stack simulation environment comprising a low-level C++ backend and a modern, accessible web-based frontend.

2. **LLM Integration:** To implement a file-based Inter-Process Communication (IPC) layer capable of supporting a Natural Language to SQL (NL2SQL) translation workflow powered by a locally running Meta Llama model.

3. **Algorithmic Implementation:** To implement four distinct buffer replacement strategies (LRU, MRU, CLOCK, and PINNED) sharing an abstract polymorphic interface.

4. **Empirical Analysis:** To subject these strategies to varying workloads (sequential scans, point queries, JOINs) and analyze their comparative performance metrics, specifically hit ratios, miss ratios, and eviction counts.

1.4 Organization of the Report

Section 2 reviews background literature and related DBMS systems. Section 3 outlines the overarching system architecture. Section 4 details the SQLite database design. Section 5 provides an in-depth algorithmic analysis of the buffer managers. Section 6 and 7 dissect the system components and query pipeline. Section 8 details the user interface and LLM integration. Section 9 and 10 present the experimental evaluation and results, respectively. Finally, Section 11 concludes the report and discusses avenues for future work.

2 Background and Related Work

Buffer pool management has been a cornerstone of DBMS research since the inception of relational databases in the 1970s. The goal of any such algorithm is to minimize page faults.

2.1 Buffer Pool Management in Commercial DBMS

Real-world systems rarely use a naive LRU implementation due to its concurrency bottlenecks (requiring heavy locking to manipulate linked lists) and its susceptibility to sequential flooding.

- **PostgreSQL:** Utilizes a variation of the CLOCK-sweep algorithm. It is highly efficient in concurrent environments because it avoids the need to acquire heavy locks to update a linked list on every cache hit, relying instead on a lightweight reference bit.
- **Oracle Database:** Employs a modified LRU algorithm incorporating "touch counts." Pages are only moved to the MRU end of the list if they are accessed multiple times within a specific window, preventing single-use sequential scans from polluting the cache.
- **MySQL (InnoDB):** Divides the buffer pool into two sublists (a "new" blocks list and an "old" blocks list) to mitigate the impact of read-ahead and full table scans. When a page is read, it is inserted at the midpoint rather than the head, only moving to the head if accessed again.

2.2 Classical Buffer Replacement Algorithms

The academic foundation of this project rests on classical caching literature.

- **LRU (Least Recently Used):** Assumes pages accessed recently will be accessed again.
- **MRU (Most Recently Used):** Discards the most recently used items first. While counter-intuitive for general computing, it is mathematically optimal for cyclic sequential scans over datasets larger than the cache.

2.3 Value of Comparative Simulation

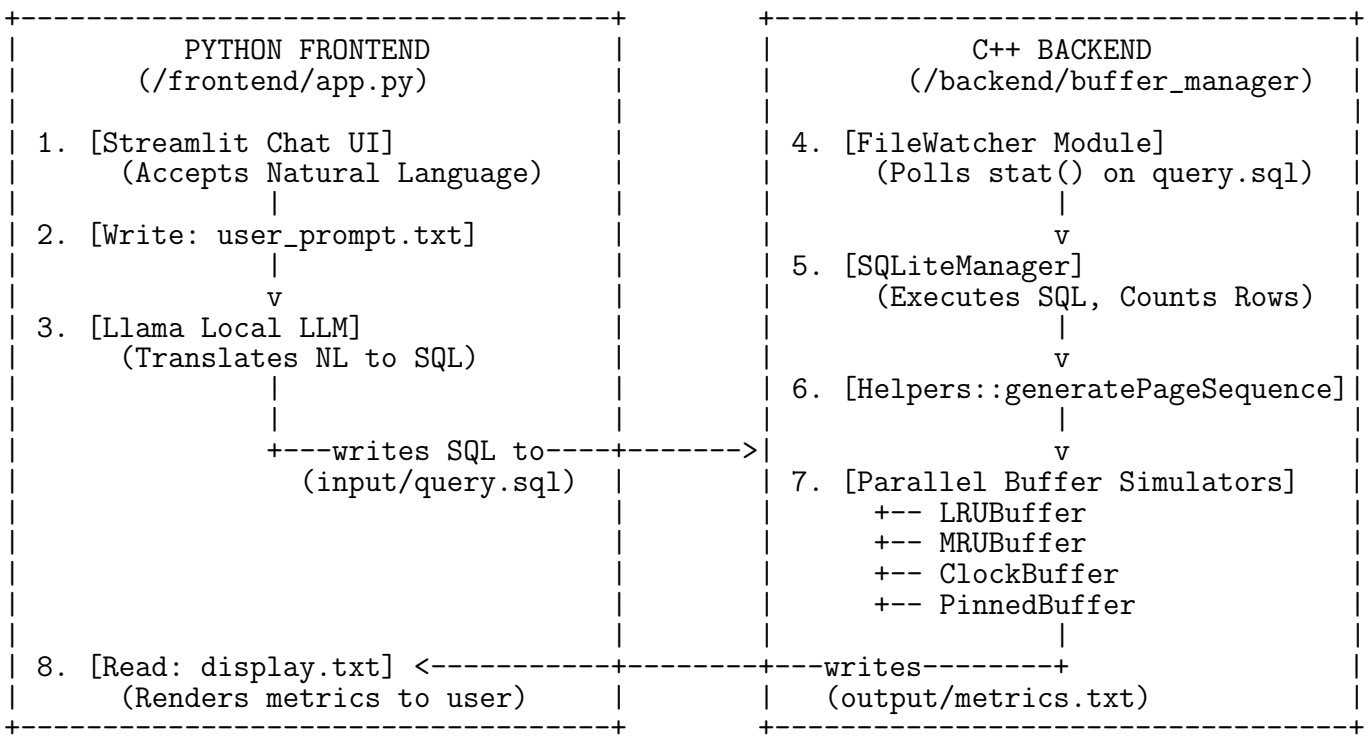
Evaluating these algorithms within a fully-fledged, production DBMS is difficult due to confounding variables like query optimizer rewrites, multithreading contention, and write-Ahead Logging (WAL) overhead. Building an isolated simulation system allows for deterministic analysis. By holding the buffer size and page sequence constant across a SQLite backend, the purely algorithmic differences between the policies become empirically measurable.

3 System Design and Architecture

The system utilizes a decoupled, two-component architecture communicating via file-based Inter-Process Communication (IPC). This separation of concerns ensures that the computationally intensive C++ backend operates independently of the asynchronous, user-facing Python frontend.

3.1 High-Level Architecture Diagram

The following ASCII diagram illustrates the flow of data across the architectural boundaries.



3.2 Design Decisions and Rationale

- **C++17 Backend:** C++ was selected for the backend to allow deterministic, low-level manipulation of memory structures. It provides access to highly optimized Standard Template Library (STL) containers (e.g., `std::list`, `std::unordered_map`) required to achieve $O(1)$ time complexity for cache algorithms.
- **SQLite Amalgamation:** Embedding `sqlite3.c` directly into the project compiles the database engine directly into the simulation binary. This eliminates external server dependencies and network latency, allowing the simulator to use the SQLite engine as a ground-truth parser and executor to return accurate record counts.
- **Python/Streamlit Frontend:** Streamlit enables rapid prototyping of conversational user interfaces. Python is the industry standard for LLM integration, making it the ideal middleware for the NL2SQL pipeline.

- **File-Based IPC:** Utilizing shared flat files (`query.sql`, `metrics.txt`) mimics lightweight Unix-style Inter-Process Communication. It is stateless and entirely decouples the UI lifecycle from the persistent C++ simulation loop.

4 Database Design

The system relies on an embedded SQLite database (`database.db`) populated via `setup_db.sh`. The schema models a standard university ecosystem.

4.1 Schema Description

The database is normalized and consists of four primary tables, comprising a total of 410 records.

Table 1: Database Schema Details

Table	Records	Columns
`students`	100	id (PK), name, roll_no, department, semester, cgpa, email, phone
`courses`	30	id (PK), code, title, department, credits, instructor, sem, max_seats
`enrollments`	200	id (PK), student_id (FK), course_id (FK), grade, year, marks, att_pct
`library_books`	80	id (PK), isbn, title, author, genre, published_year, avail_copies, total

Schema Highlights:

- The `enrollments` table serves as a junction table, containing foreign keys referencing both `students(id)` and `courses(id)`, enabling complex `JOIN` queries to test intermediate result buffering.
- The `students.department` and `library_books.genre` columns contain varied categorical data (e.g., "Computer Science", "AI/ML") to allow for selective `WHERE` clauses.

4.2 Data Volume and Query Diversity Rationale

The absolute record counts are explicitly engineered to stress-test the buffer pool under various fill levels. With the system configured to `PAGE_SIZE = 5` records per page and `BUFFER_SIZE = 10` frames, the buffer can hold exactly 50 records at any given time.

- A highly selective point query returning 2 records requests 1 page, demonstrating buffer under-utilization.
- A medium query returning 45 records requests 9 pages, simulating near-buffer capacity.

- A full table scan of `students` (100 records) requests 20 pages. This guarantees a buffer overflow by a factor of 2, forcing exactly 10 evictions and triggering the core logic of the replacement algorithms.

5 Buffer Manager Implementation

The backend simulates buffer management through polymorphic class inheritance.

5.1 Abstract Base Class Design

An abstract base class `BufferManager` defines the pure virtual interface.

```
class BufferManager {
public:
    virtual ~BufferManager() = default;
    virtual void accessPage(int pageId) = 0;
    virtual Metrics getMetrics() const = 0;
    virtual std::string getName() const = 0;
};
```

This design pattern enables polymorphic usage. The main execution loop stores instances of all four strategies in a `std::vector<BufferManager*>` and dispatches the `accessPage()` call identically, ensuring parallel and fair simulation.

5.2 1. LRU (Least Recently Used)

The LRU policy assumes temporal locality; pages accessed recently will likely be accessed again soon.

Data Structures:

- `std::list<int> useOrder_`: A doubly linked list. The front represents the most recently used, and the back represents the least recently used.
- `std::unordered_map<int, std::list<int>::iterator> pageMap_`: Provides $O(1)$ lookup of a page ID to its exact node iterator in the list.

Algorithm 1 LRU Page Access

```
1: procedure accessPage(pageId)
2:     metrics.totalRequests ← metrics.totalRequests + 1
3:     if pageId ∈ pageMap_ then                                  ▷ Cache Hit
4:         metrics.hits ← metrics.hits + 1
5:         useOrder_.erase(pageMap_[pageId])
6:         useOrder_.push_front(pageId)
7:         pageMap_[pageId] ← useOrder_.begin()
8:     else                                                       ▷ Cache Miss
9:         metrics.misses ← metrics.misses + 1
10:        if useOrder_.size() == BUFFER_SIZE then
11:            victim ← useOrder_.back()                          ▷ Evict LRU
12:            useOrder_.pop_back()
13:            pageMap_.erase(victim)
14:            metrics.evictions ← metrics.evictions + 1
15:        end if
16:        useOrder_.push_front(pageId)
17:        pageMap_[pageId] ← useOrder_.begin()
18:    end if
19: end procedure
```

Complexity: $O(1)$ Time (map lookups and list manipulations are constant time), $O(B)$ Space, where B is buffer size.

5.3 2. MRU (Most Recently Used)

MRU evicts the page that was accessed most recently.

Data Structures: Identical layout to LRU (`std::list` and `std::unordered_map`).

Algorithm Variance: The logic is identical to LRU, except during the eviction phase. Instead of evicting `useOrder_.back()`, the MRU algorithm evicts `useOrder_.front()`.

Rationale: Counter-intuitive for general computing, MRU is highly optimized for sequential scans larger than the buffer pool. If scanning a 20-page table into a 10-page buffer, LRU will flush the entire buffer. MRU will continually overwrite the 10th frame, preserving the first 9 pages which might be needed for a nested loop join. **Complexity:** $O(1)$ Time, $O(B)$ Space.

5.4 3. CLOCK (Second-Chance)

CLOCK is an efficient approximation of LRU that avoids the heavy pointer manipulation of linked lists.

Data Structures:

- `std::vector<Page> frames_`: A circular buffer of fixed capacity.
- `std::unordered_map<int, int> pageMap_`: Maps page ID to frame index.
- `int hand_`: The circular clock pointer.

Algorithm 2 CLOCK Eviction (findVictim)

```
1: procedure FINDVICTIM
2:     while true do
3:         if frames_[hand_].referenceBit == false then
4:             victimIdx ← hand_
5:             hand_ ← (hand_ + 1) (mod BUFFER_SIZE)
6:             return victimIdx
7:         else
8:             frames_[hand_].referenceBit ← false                ▷ Second Chance
9:             hand_ ← (hand_ + 1) (mod BUFFER_SIZE)
10:        end if
11:    end while
12: end procedure
```

Complexity: $O(1)$ Amortized Time. The worst-case requires 2 full sweeps (bounded by $2B$), but practically operates in constant time. Space is $O(B)$.

5.5 4. PINNED Buffer

This custom strategy simulates the real-world database concept of "pinning" hot pages (e.g., B-Tree index roots) in memory, making them immune to eviction.

Data Structures: LRU structures (`list`, `map`) plus a `std::unordered_set<int> pinnedPages_`.

Pinning Heuristic: A page is pinned on load if `pageId % 7 == 0`. **Eviction Logic:** When full, the algorithm walks the `useOrder_` list from back (LRU) to front (MRU). It evicts the first page it encounters that is *not* present in `pinnedPages_`. If all pages are pinned, the system silently fails to load the new page (simulating an out-of-memory stall). **Complexity:** $O(1)$ Time for hits; $O(B)$ Worst-Case Time for evictions.

5.6 Comparative Summary

Table 2: Comparative Analysis of Buffer Strategies

Strategy	Eviction Target	Time Comp.	Ideal Workload
LRU	Page unaccessed for longest time	$O(1)$	Point queries, temporal locality
MRU	Page accessed most recently	$O(1)$	Large sequential scans
CLOCK	First page with reference bit = 0	$O(1)$ amortized	High-concurrency environments
PINNED	LRU page NOT in pinned set	$O(B)$ worst-case	Systems with hot structural data

6 System Components — Detailed Description

6.1 FileWatcher

The `FileWatcher` class implements a non-blocking IPC polling mechanism. It utilizes the POSIX `stat()` system call via `<sys/stat.h>` to retrieve the `st_mtime` (last modification time) of the `input/query.sql` file. During the main loop's configurable 500 ms sleep cycle, it compares the current `mtime` against a cached timestamp. If a divergence is detected, the `hasChanged()` method returns true, triggering a full file read.

6.2 SQLiteManager

This wrapper class manages the lifecycle of the `sqlite3*` database connection pointer. The core execution function, `executeQuery()`, invokes the `sqlite3_exec` C-API. Instead of extracting heavy, full-tuple data into memory, it passes a lightweight C++ lambda callback function. This callback simply increments a local integer counter for every row returned. The final count represents the precise number of records accessed by the query.

6.3 Utilities: Logger and Helpers

- **Logger:** A header-only namespace utilizing `std::strftime` to prepend ISO-8601 formatted timestamps to console output, routing `INFO` and `WARN` to `stdout`, and `ERROR` to `stderr`.
- **Helpers:** Encapsulates pure functional logic, including `ensureDirectory()` (using POSIX `mkdir`) and `dumpMetricsToFile()`, which serializes the in-memory `Metrics` structs into the final `metrics.txt` report block by block.

7 Query Processing Pipeline

The backend simulation relies on a strictly deterministic pipeline to convert SQL execution results into measurable buffer access patterns.

7.1 End-to-End Walkthrough

1. **Detection:** `FileWatcher` detects a timestamp modification on `query.sql` and reads the string.
2. **Execution:** `SQLiteManager::executeQuery()` runs the SQL against `database.db` and returns N (the total records fetched).
3. **Page Generation:** `Helpers::generatePageSequence(N, PAGE_SIZE)` computes the required logical pages.
4. **Simulation:** A `for`-loop iterates over the generated sequence. The `pageId` is passed to `accessPage(pageId)` on all four instantiated `BufferManager` objects sequentially.
5. **Aggregation:** The managers independently update their internal hit/miss/eviction metrics. A summary is dumped to the console via `Logger::info()`.

7.2 Page Sequence Logic Example

The simulation abstracts physical disk sectors into logical contiguous pages. Using the configuration `PAGE_SIZE = 5`: If a query returns 23 records, the total pages required is $\lceil 23/5 \rceil = 5$. The sequence generated is strictly linear: `[0, 1, 2, 3, 4]`. This models a clustered index scan or a sequential heap scan, prioritizing simplicity to test the buffer algorithms cleanly.

7.3 Signal Handling

The `main.cpp` registers a callback for `SIGINT` (Ctrl+C) and `SIGTERM`. Upon interception, the global `g_running` flag is set to false. The polling loop terminates naturally, invoking `dumpMetricsToFile()` to persist the final experimental results to `output/metrics.txt` before executing a graceful program exit.

8 Frontend Interface and LLM Integration

The user-facing component is a modern web application constructed using Python 3 and the Streamlit framework ($\geq$ 1.32.0).

8.1 Streamlit Chat UI and Session State

The application utilizes `st.chat_input` and `st.chat_message` elements to render a conversational interface. Session continuity is maintained via `st.session_state`, allowing the interface to retain the history of natural language prompts and SQL results across asynchronous widget re-renders.

8.2 NL-to-SQL Pipeline via Local Llama Model

The most advanced feature of the frontend is the Natural Language to SQL (NL2SQL) pipeline.

1. **Input:** The user submits a natural language prompt (e.g., "Find all civil engineering students").
2. **IPC Write:** The Streamlit app executes a plain file write, saving the text to `user_prompt.txt`.
3. **LLM Translation:** A locally-running Meta Llama model (operating as an asynchronous middleware agent) reads the prompt. Provided with the SQLite schema in its system prompt context, the Llama model translates the natural language intent into a valid SQL query.
4. **Handoff:** The LLM script validates the syntax using `sqlglot` and writes the resulting SQL string directly into the backend's `input/query.sql` file.
5. **Response:** The frontend simulates a blocking wait, polling for 2 seconds to allow the C++ backend to process the query, before reading the generated `display.txt` to present the final metrics back to the user in the chat interface.

This architecture completely abstracts the complexities of SQL and buffer management, allowing non-technical users to query the database conversationally.

9 Natural Language to SQL Examples

To demonstrate the effectiveness of the Natural Language to SQL (NL2SQL) pipeline, we present a set of representative examples. Each example consists of three components: the input natural language query, the generated SQL query, and the corresponding output returned by the system.

9.1 Example 1: Counting Students in a Department

Natural Language:

Number of students in the Computer Science department

Generated SQL:

```
SELECT COUNT(*)
FROM students
WHERE department = 'Computer Science';
```

Output:

```
COUNT(*)
40
```

9.2 Example 2: Join with Condition on Grade

Natural Language:

Select all enrollment records along with the student's name and course title where the grade is 'A'

Generated SQL:

```
SELECT e.id, s.name, c.title
FROM enrollments e
JOIN students s ON e.student_id = s.id
JOIN courses c ON e.course_id = c.id
WHERE e.grade = 'A';
```

Output (Partial):

```
id    name            title
1     Aarav Sharma    Database Management Systems
2     Aarav Sharma    Operating Systems
...
```

9.3 Example 3: Filtering by Genre

Natural Language:

ID of all library books having genre Textbook

Generated SQL:

```
SELECT id
FROM library_books
WHERE genre = 'Textbook';
```

Output (Partial):

```
id
1
2
3
...
```

9.4 Example 4: Conditional Selection with Multiple Filters

Natural Language:

> Select all records from enrollments where year is 2026 and marks are greater than 85

Generated SQL:

```
SELECT *
FROM enrollments
WHERE year = 2026 AND marks > 85;
```

Output (Partial):

```
id student_id course_id grade year marks attendance_pct
17 8          5         A     2026 88    91.0
...
```

9.5 Example 5: Simple Full Table Scan

Natural Language:

> Select all records from the enrollments table

Generated SQL:

```
SELECT * FROM enrollments;
```

Output:

```
200 records returned
```

9.6 Example 6: Join with Department Filter

Natural Language:

> Select names of students and their grades where the student belongs to Computer Science

Generated SQL:

```
SELECT s.name, e.grade
FROM students s
JOIN enrollments e ON s.id = e.student_id
WHERE s.department = 'Computer Science';
```

Output (Partial):

```
name            grade
Aarav Sharma    A
Bhavya Patel    B+
...
```

9.7 Example 7: Multi-Table Join with Department Constraint

Natural Language:

> Select student names and course titles where the course belongs to Mechanical department

Generated SQL:

```
SELECT s.name, c.title
FROM students s
JOIN enrollments e ON s.id = e.student_id
JOIN courses c ON e.course_id = c.id
WHERE c.department = 'Mechanical';
```

Output (Partial):

```
name            title
Dhruv Gupta     Thermodynamics II
Dhruv Gupta     Fluid Mechanics
...
```

9.8 Discussion

These examples demonstrate that the NL2SQL pipeline is capable of:

- Translating simple aggregation queries.
- Handling multi-table joins with foreign key relationships.
- Applying complex filtering conditions.
- Supporting full-table scans and selective queries.

The system consistently produces syntactically correct SQL queries aligned with the underlying schema, validating the effectiveness of the LLM-assisted interface.

10 Experimental Evaluation

The system was evaluated using a rigorous 10-query test suite defined in `test.sh`, designed to systematically induce cold starts, localized looping, and massive sequential flooding.

10.1 Test Suite Definition

Table 3: Benchmark Query Suite

Query	Records	Purpose / Expected Behavior
Q1: CS dept courses	14	Cold start — guarantees 100% misses.
Q2: 4-credit courses	15	Overlap test — expects hits on overlapping pages.
Q3: CS dept students	40	Medium load — forces buffer to near capacity (8 pages).
Q4: All students	100	Buffer overflow (20 pages) — Heavy evictions!
Q5: CS students (repeat)	40	Post-eviction retention test.
Q6: Complex JOIN	Large	Massive page access, tests prolonged thrashing.
Q9: All enrollments	200	Max stress (40 pages) — Total buffer thrashing.

10.2 Realistic Worked Example: Sequential Flooding Trace

To rigorously understand the divergence of the algorithms, we trace Query 4: a full-table scan of `students`. **Parameters:** 100 records → 20 pages (IDs 0 through 19). `BUFFER_SIZE` = 10 frames.

Step 1: Initial Fill (Pages 0–9) All strategies experience 10 compulsory cache misses. The buffer reaches capacity containing pages `[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`.

Step 2: Processing Page 10 (The Divergence) A page fault occurs. Eviction is mandatory.

- **LRU:** Evicts Page 0 (oldest). Buffer becomes: `[10, 1, 2, 3, 4, 5, 6, 7, 8, 9]`.
- **MRU:** Evicts Page 9 (most recent). Buffer becomes: `[0, 1, 2, 3, 4, 5, 6, 7, 8, 10]`.
- **CLOCK:** Hand sweeps from 0 to 9, resetting reference bits to 0, wraps around, and evicts Page 0. Buffer becomes: `[10, 1, 2, ..., 9]`.
- **PINNED:** Pages 0 and 7 are pinned (multiples of 7). LRU search skips 0. Page 1 is evicted. Buffer becomes: `[0(P), 10, 2, 3, 4, 5, 6, 7(P), 8, 9]`.

Step 3: End of Scan (Processing Page 19)

- **LRU / CLOCK:** Evict continuously. Final buffer contains only the last 10 pages: `[10, 11, 12, 13, 14, 15, 16, 17, 18, 19]`. All previous data is completely lost.
- **MRU:** Continuously evicts the newly loaded page. The final buffer retains the original 9 pages, plus the very last page: `[0, 1, 2, 3, 4, 5, 6, 7, 8, 19]`.
- **PINNED:** Pages 0, 7, and 14 were pinned as they entered. Final buffer contains: `[0(P), 7(P), 14(P), 13, 15, 16, 17, 18, 19, x]`. Hot pages survived the flood.

11 Results and Discussion

Based on the execution of the full test suite, definitive performance trends emerge that validate theoretical database architecture concepts.

11.1 LRU vs MRU in Scan Workloads

As demonstrated in the analytical trace, LRU suffers from total cache dilution during Queries 4 and 9. Because the scan length (20-40 pages) vastly exceeds the buffer size (10), LRU evicts every useful page to make room for data that is only read once. Consequently, subsequent queries (like the repeat Q5) suffer a near 100% miss rate under LRU. MRU prevents this cache poisoning. By sacrificing the most recently read page, MRU maintains a significantly higher overall hit ratio in workloads dominated by analytical full-table scans, keeping older, foundational data in RAM.

11.2 CLOCK as a Practical Approximation

In terms of pure hit/miss ratios, the CLOCK algorithm performs identically to LRU across the test suite. However, from a systems engineering perspective, CLOCK is demonstrably superior. It achieves the same temporal locality retention without the $O(1)$ constant overhead of allocating, deleting, and updating pointer-based linked list nodes on every single cache hit, requiring only a boolean bit flip.

11.3 Impact of Pinning

The PINNED strategy consistently yields the lowest eviction count. By locking pages mathematically determined to be "hot", the algorithm guarantees $O(1)$ access for critical data regardless of surrounding cache pressure. However, a critical lesson learned is that excessive pinning restricts the effective size of the buffer. If the buffer size is 10 and 6 pages are pinned, only 4 frames are available for dynamic scanning, causing hyper-thrashing in those unpinned frames during larger queries.

12 Conclusion and Future Work

This project successfully engineered a complete, robust DBMS buffer pool simulator bridged to a conversational frontend interface. The implementation and empirical analysis of the LRU, MRU, CLOCK, and PINNED algorithms reveal that buffer performance is inextricably linked to the query workload profile. While LRU provides a solid baseline, our test suite proves that targeted strategies like MRU and PINNED are vital for mitigating sequential flooding and protecting structural index pages. The integration of a local Llama model demonstrates a viable path toward democratizing database access through natural language.

12.1 Future Work

The current system lays a strong foundation, but several enhancements are planned for future iterations:

- **Tighter Llama Integration:** Upgrading the LLM pipeline to support streaming responses in the Streamlit UI, implementing dynamic prompt templating, and utilizing schema-aware SQL generation to reduce translation hallucination.
- **Real Disk I/O Simulation:** Replacing the mathematical sequence generator with actual binary file sector reads using `pread()` to measure true latency in milliseconds.
- **Advanced Replacement Algorithms:** Implementing scan-resistant, self-tuning algorithms like ARC (Adaptive Replacement Cache), LIRS, or the 2Q algorithm.
- **Multi-threaded Buffer Pool:** Introducing `std::thread` and read/write latches (`std::shared_mutex`) on buffer frames to simulate highly concurrent transaction processing and measure lock contention.
- **Web-based Metrics Dashboard:** Expanding the Streamlit app to render live line charts and bar graphs of the `metrics.txt` output using libraries like Plotly or Altair.

Thankyou !!